Relatório de Auditoria de Código

Módulo Cartão Benefícios — abas Vendas, Convênios e Dependentes

Projeto	ClinicaOS / Health Hub Pro (patientpal-secure)
Módulo auditado	Cartão Benefícios → Vendas, Convênios, Dependentes
Data	13/07/2026
Branch	main (commit 5730919b, pós git pull --ff-only)
Escopo	Leitura de código (sem alterações aplicadas) + tsc --noEmit + eslint

Resumo executivo

Foram revisados os três arquivos que implementam as abas Vendas, Convênios e Dependentes do módulo Cartão Benefícios: `contratos-page.tsx`, `app.cartao-beneficios.convenios.tsx` e `app.cartao-beneficios.dependentes.tsx`, além da lib compartilhada `contrato-dependentes.ts`.

Compilação e sintaxe: os três arquivos compilam sem erros de tipo (`tsc --noEmit`). Os 8 erros de TypeScript existentes no projeto estão todos em outros módulos (médicos, hr-contratos, orçamentos, atendimento-ia), fora do escopo desta auditoria. O ESLint não aponta erro de lógica nestes arquivos — apenas ~712 avisos de formatação Prettier em `dependentes.tsx`, que não foi rodado por `npm run format`.

Achados relevantes: 4 problemas de atomicidade (operações multi-etapa sem transação, risco de estado inconsistente em falha parcial) e 4 problemas de lógica, sendo um deles (L1) uma divergência de valor no card "Recebido" que soma a taxa de adesão em duplicidade. Nenhuma alteração foi aplicada ao código — este relatório é apenas de leitura/análise.

Categoria	Alta	Média	Baixa	Total
Atomicidade (transações)	1	3	0	4
Lógica de negócio	1	1	2	4
Multi-tenant / outros	0	1	2	3

Legenda de severidade: **ALTA** — risco de perda/inconsistência de dados financeiros; **MÉDIA** — comportamento incorreto visível ao usuário, sem perda de dados; **BAIXA** — dívida técnica ou melhoria pontual.

1. Erros de atomicidade

Estas operações fazem múltiplas chamadas separadas ao Supabase (DELETE, depois INSERT, depois UPDATE) em vez de uma única RPC transacional no Postgres. Se qualquer etapa falhar no meio do caminho, o registro fica em estado parcial — o mesmo problema que motivou a migração do fluxo de criação de contrato para a RPC `criar_contrato_assinatura` (ver comentário em `contratos-page.tsx:1021`).

A — Regeneração de parcelas pode apagar parcelas sem recriar

ALTA

`src/components/pages/contratos-page.tsx — função regerarComPagas(), linhas 1810–1853`

A função apaga (`DELETE`) todas as parcelas pendentes do contrato e, em seguida, insere (`INSERT`) as 12 novas parcelas em uma chamada separada. Se o INSERT falhar após o DELETE ter sido concluído com sucesso (erro de rede, timeout, violação de constraint), o contrato fica **sem nenhuma parcela pendente** — perda de dados financeiros que só é sinalizada por um toast de erro, sem rollback automático.

Correção recomendada: Mover a lógica para uma RPC no Postgres que execute DELETE + INSERT dentro de uma única transação, no mesmo padrão de `criar_contrato_assinatura`.

B — Salvar dados financeiros pode deixar contrato sem parcelas futuras

MÉDIA

`src/components/pages/contratos-page.tsx — função salvarDadosFinanceiros(), linhas 1906–1979`

Sequência de três operações independentes: UPDATE do contrato, DELETE das parcelas pendentes futuras e INSERT de 12 novas parcelas. Uma falha no INSERT deixa o contrato atualizado mas sem parcelas futuras geradas. O código já trata parcialmente esse caso (mensagem "dados salvos, mas falha ao gerar parcelas"), mas o estado inconsistente permanece no banco até correção manual.

Correção recomendada: Envolver as três etapas em uma RPC transacional; manter a UI apenas como cliente da RPC.

C — Salvar convênio pode deixar faixas de preço ou benefícios ausentes

MÉDIA

`src/routes/_authenticated/app.cartao-beneficios.convenios.tsx — função save(), linhas 336–462`

Até 5 requests separados: upsert do convênio, DELETE + INSERT das faixas de preço e DELETE + INSERT dos benefícios. Uma falha no INSERT das faixas (por exemplo) deixa o convênio salvo mas **sem nenhuma faixa de preço** — a coluna "A partir de" na listagem fica em "—" e novas vendas desse convênio ficam sem valor mensal definido.

Correção recomendada: Consolidar em uma RPC transacional (upsert convênio + replace faixas + replace benefícios).

D — Edição administrativa do contrato e recálculo de parcelas são etapas independentes

MÉDIA

`src/components/pages/contratos-page.tsx — função salvarContratoAdmin(), linhas 1717–1798 (Promise.all em 1775)`

O UPDATE do contrato (valor mensal, convênio, paciente) e o recálculo em lote das parcelas em aberto (via `Promise.all`) são chamadas independentes. Se o UPDATE do contrato for bem-sucedido mas uma das atualizações de parcela falhar (o `Promise.all` não é atômico — outras parcelas já atualizadas permanecem alteradas), o valor mensal do contrato diverge do valor das parcelas em aberto.

Correção recomendada: Substituir por uma única RPC que atualize contrato e parcelas em aberto na mesma transação.

2. Erros de lógica de negócio

L1 — Card "Recebido" soma a taxa de adesão em duplicidade

ALTA

`src/components/pages/contratos-page.tsx — linhas 2246–2247 (cálculo) e 3247–3272 (origem do lançamento)`

`totalPagoMens` soma o valor de **todas** as linhas pagas de `mens`, incluindo a linha de adesão (`numero_parcela = 0`, cujo valor é a própria taxa). Paralelamente, `extraRecebido.total` soma os lançamentos financeiros do paciente filtrados por `paciente_id` — e o lançamento da taxa de adesão grava esse campo (linha 3259), então também entra nessa soma. Resultado: em qualquer contrato com taxa de adesão paga, o card **"Recebido" exibe a taxa contada duas vezes**. A contagem de número de parcelas ("Pagas X/Y") não tem esse problema — apenas o valor em R$.

Correção recomendada: Excluir a linha de adesão de `totalPagoMens` (`mens.filter(m => m.status === "pago" && !isAdesao(m))`) ou remover a taxa de adesão do cálculo de `extraRecebido`.

L2 — Contagem de parcelas diverge entre a lista e o detalhe do contrato

MÉDIA

`src/components/pages/contratos-page.tsx — linhas 240–256 (lista, dentro de load()) vs. linha 2244 (detalhe)`

Na tela de listagem, `parcAgg` conta **todas** as linhas de `contrato_mensalidades`, incluindo a linha de adesão. No detalhe do contrato, `mensalidades = mens.filter(m => !isAdesao(m))` exclui essa linha explicitamente. Um contrato com taxa de adesão e 12 mensalidades mostra **"0/13" na lista** mas **"0/12" no detalhe** — número diferente para o mesmo contrato, dependendo de onde o operador está olhando.

Correção recomendada: Filtrar `numero_parcela != 0` na query de agregação da lista, alinhando com o critério já usado no detalhe.

L3 — Botão "Pagar" nunca desabilita em contrato cancelado

BAIXA

`src/components/pages/contratos-page.tsx — linhas 1658 e 2741`

O botão de pagar parcela usa `disabled={cancelado && !isAdmin}`, mas `isAdmin` está fixado como `const isAdmin = true` na linha 1658 (comentário indica que foi liberado para todos os perfis). Como resultado, a condição é sempre falsa e o botão **nunca desabilita**, mesmo com o contrato cancelado — é possível registrar pagamento de parcela em contrato já cancelado.

Correção recomendada: Possível regra de negócio — validar com a equipe da clínica se contratos cancelados devem permitir cobrança de débito residual ou se o botão deve bloquear.

L4 — Cancelamento de contrato não trata as parcelas em aberto

BAIXA

`src/components/pages/contratos-page.tsx — função confirmarCancelamento(), linhas 1981–2008`

Ao cancelar, apenas os campos `status`, `cancelado_em` e `cancelamento_motivo` do contrato são gravados. As parcelas pendentes continuam existindo sem alteração e seguem somando no card "A receber" do contrato — combinado com L3, um contrato cancelado pode continuar acumulando cobrança ativa.

Correção recomendada: Possível regra de negócio — validar com a equipe da clínica se o cancelamento deve cancelar/baixar automaticamente as parcelas pendentes.

---

3. Multi-tenant e pontos menores

M1 — Busca de paciente no modal de Dependentes sem filtro de clínica

MÉDIA

`src/routes/_authenticated/app.cartao-beneficios.dependentes.tsx — linha 295`

O `PatientSearchInput` usado no modal "Adicionar dependente" não recebe `clinicaIdsOverride`, diferente dos outros dois pontos de busca de paciente em `contratos-page.tsx` (linhas 1382 e 3359), que restringem a clínica atual. Como `incluirDependenteContrato()` (em `src/lib/contrato-dependentes.ts`) não valida a clínica do paciente selecionado, é teoricamente possível incluir como dependente um paciente cadastrado em **outra clínica**. Alinhado ao gap conhecido do projeto descrito em `.claude/rules/seguranca-lgpd.md` (item 6): autorização precisa estar no servidor/RLS, não apenas na UI.

Correção recomendada: Adicionar `clinicaIdsOverride={{[clinicaAtual.clinica_id]}}` ao PatientSearchInput e, idealmente, validar a clínica do paciente também dentro de `incluirDependenteContrato()` como defesa em profundidade.

M2 — Arquivo de Dependentes fora do padrão de formatação do projeto

BAIXA

`src/routes/_authenticated/app.cartao-beneficios.dependentes.tsx — arquivo inteiro`

O ESLint reporta ~712 avisos de `prettier/prettier` neste arquivo (quebras de linha/indentação divergentes do Prettier configurado). Não são erros de lógica, apenas formatação — o arquivo não passou por `npm run format` desde a última edição.

Correção recomendada: Rodar `npm run format` — atenção: isso reformata o arquivo inteiro, gerando um diff grande só de formatação; considerar um commit isolado para não misturar com mudanças funcionais.

M3 — Formatacão de moeda duplicada localmente

BAIXA

`src/components/pages/contratos-page.tsx — linha 67 (const BRL) e app.cartao-beneficios.convenios.tsx — linha 510`

Os três arquivos implementam sua própria formatação de moeda em vez de reutilizar `src/lib/financeiro/format.ts`. Esta é uma dívida técnica já catalogada no `CLAUDE.md` do projeto (~18 reimplementações conhecidas em todo o código) — não foi introduzida por este módulo, apenas confirmada aqui.

Correção recomendada: Migrar para o util central quando este módulo for tocado por outra tarefa, conforme já orientado nas regras de frontend do projeto.

Anexo — Comandos de validação executados

O projeto não possui script `test` nem framework de testes automatizados configurado (confirmado em `package.json`). As validações abaixo foram executadas manualmente sobre a árvore de trabalho, restrita aos três arquivos das abas auditadas.

Comando	Resultado
`tsc --noEmit`	0 erros nos 3 arquivos auditados (8 erros pré-existentes em outros módulos, fora do escopo)
`eslint (nos 4 arquivos)`	0 erros de lógica/regra; ~712 avisos de formatação Prettier em dependentes.tsx
`npm run test`	Não aplicável — sem framework de testes configurado no projeto

Este relatório é somente de leitura/análise — nenhuma alteração de código foi aplicada durante a auditoria. Achados marcados como "possível regra de negócio" (L3, L4) requerem validação com a equipe da clínica antes de qualquer correção, conforme as regras do projeto.